DriftGuard

Online drift-aware visual object tracking

Custom Kernelized Correlation Filter + PSR failure detection + frozen-model reacquisition + verified recovery

Technical report: method, implementation, datasets, experiments, failed attempts and results

Project	DriftGuard — lightweight single-object tracking
Core tracker	Custom NumPy FFT KCF (no cv2.TrackerKCF_create)
Evaluation sets	OpenCV vtest.avi (no GT); OTB-2015 Jogging (official GT)
Key result	OTB Jogging aggregate success@0.5 = 0.984; baseline = 0.612
Report version	1.0 — 10 November 2026

Executive verdict
The original vtest-only result was not a defensible accuracy evaluation. After adding official OTB ground truth and fixing the recovery state machine, DriftGuard reaches 98.4% aggregate success and 100% precision@20px on the two Jogging targets, while the always-updating baseline reaches 61.2% success and 61.6% precision. Both occlusion episodes are recovered.

Executive Summary

DriftGuard is a lightweight, CPU-oriented single-object tracker that wraps a custom Kernelized Correlation Filter (KCF) with explicit confidence monitoring. The project was motivated by a practical weakness of plain KCF: when the target is occluded or leaves the search region, the filter can either drift silently or adapt its model to the occluder. The system therefore exposes the raw correlation response map, computes the Peak-to-Sidelobe Ratio (PSR), freezes model updates when confidence collapses, and attempts recovery through a frozen-filter reacquisition followed by verified template matching.

The implementation is deliberately self-contained. OpenCV's Python KCF wrapper does not expose the response map reliably, so the tracker is a NumPy FFT implementation using grayscale + Lab chroma features. It supports an initial bounding box, per-frame response maps, freeze-aware updates, multi-template recovery, histogram/SSIM verification, rollback, and OTB-style evaluation.

What changed after the first critical verdict
The first vtest-only result was misleading: 44.1% PSR-healthy frames, one confirmed recovery and 391 LOST frames, with no ground truth. I added the OTB-2015 Jogging sequence with official annotations, then changed recovery from immediate template re-initialisation to **frozen-model reacquisition first**. This fixed a false pole re-initialisation on Jogging-2 and produced a defensible quantitative comparison against baseline KCF.

Headline results

Benchmark	Metric	DriftGuard	Baseline KCF
OTB Jogging aggregate (614 GT frames)	Success at IoU 0.5	0.984	0.612
OTB Jogging aggregate	Success AUC	0.773	0.463
OTB Jogging aggregate	Mean IoU	0.776	0.461
OTB Jogging aggregate	Precision at 20 px	1.000	0.616
Jogging-1 (occluded target)	Success at IoU 0.5	0.967	0.225
Jogging-2 (pole pass)	Success at IoU 0.5	1.000	1.000
vtest.avi (no GT)	PSR-healthy proxy frames	350 / 794	691 / 794

The rest of this report documents the method, implementation, data acquisition, metric definitions, every measured result, and the failed attempts that shaped the final design. The vtest numbers are reported only as PSR/state proxies because that clip has no authoritative per-frame labels.

Contents

Executive Summary 2

Headline results 2

1. Problem Statement and Scope 5

Contributions 5

2. Datasets and Ground Truth 6

2.1 OpenCV vtest.avi 6

2.2 OTB-2015 Jogging (true ground truth) 6

2.3 Ground-truth CSV format 6

2.4 Data acquisition failures and limitations 7

3. Method 8

3.1 Runtime pipeline 8

3.2 Custom KCF core 8

3.3 PSR drift detection 8

3.4 Freeze, frozen reacquisition and template recovery 9

3.5 Baseline tracker 9

4. Implementation 10

4.1 Classes and responsibilities 10

4.2 Important implementation details 10

4.3 Parameters used in the reported experiments 11

4.4 Key code excerpt: frozen reacquisition 11

4.5 Command-line interface and outputs 11

5. Experimental Protocol 12

5.1 One-pass evaluation 12

5.2 Metrics 12

5.3 Environment 12

6. Results . . . 13

6.1 OTB Jogging accuracy . . . 13

6.2 Recovery behaviour . . . 15

6.3 Qualitative OTB frames . . . 15

6.4 PSR behaviour . . . 17

6.5 vtest.avi proxy results . . . 17

6.6 Computational efficiency . . . 18

7. Failed Attempts and Design Iteration . . . 20

7.1 Missing vtest ground truth . . . 20

7.2 Response-map sign inversion . . . 20

7.3 Sign/pole model contamination . . . 20

7.4 False recovery on Jogging-2 . . . 20

7.5 The fixed-delay trap . . . 20

7.6 Other rejected or deferred ideas . . . 21

7.7 Remaining limitations . . . 21

8. Conclusion . . . 22

9. Reproducibility . . . 23

9.1 Installation . . . 23

9.2 Original vtest demo . . . 23

9.3 OTB Jogging benchmark . . . 23

9.4 Tests . . . 23

9.5 Output manifest . . . 23

Appendix A. Metric Definitions . . . 24

Appendix B. Complete Result Summary . . . 25

Appendix C. References and Data Attribution . . . 26

1. Problem Statement and Scope

Visual object tracking starts from a single bounding box and must locate the same object in subsequent frames. In surveillance footage the target can be partially or fully occluded, can change pose and scale, can leave the field of view, and can be confused with other pedestrians. A standard KCF tracker is fast and accurate on short, unoccluded sequences, but it has two failure modes relevant here:

- **Silent drift:** the response peak moves to background or another object while the filter remains self-confident.
- **Model contamination:** the update step learns the occluder, so the tracker cannot recover even after the target reappears.

The project requirement was to build a lightweight real-time tracker around KCF, monitor confidence via PSR, freeze updates below a threshold, re-detect and verify the target, and compare against an always-updating baseline. The original demo uses OpenCV's `vtest.avi` with initial box `(638, 238, 48, 86)`.

The scope of this report is the implemented system, the datasets actually used, quantitative success/precision results where ground truth exists, computational efficiency, and a transparent record of approaches that failed. It is not a claim of state-of-the-art performance on OTB100, LaSOT or GOT-10k.

Contributions

- A self-contained NumPy KCF that exposes the full correlation response map and supports snapshot/rollback, avoiding the response-map limitation of OpenCV's Python KCF wrapper.
- A PSR-based freeze policy that prevents model contamination when correlation confidence collapses.
- A recovery pipeline with frozen-KCF reacquisition, multi-template/multi-scale NMS, colour/SSIM verification, motion gating, confirmation and rollback.
- A true OTB Jogging ground-truth benchmark with success, success-AUC, mean IoU, precision, precision-AUC and recovery metrics, plus optional GT curves.
- A transparent failure log covering sign inversion, pole contamination, false recovery, fixed-delay trade-offs and the limits of PSR-only evaluation.

2. Datasets and Ground Truth

2.1 OpenCV vtest.avi

Source: OpenCV sample data, `https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/vtest.avi`. The clip is 768x576, 10 FPS, 795 frames, and shows pedestrians on a paved area. The task supplies only the first box; no per-frame ground-truth file is distributed with the OpenCV sample. This is the sequence used for the original visual demo.

Property	Value
Resolution / FPS	768 x 576 @ 10 FPS
Frames	795
Initial box	(x, y, w, h) = (638, 238, 48, 86)
Ground truth	None in the OpenCV sample
Use in this report	Qualitative demo and PSR/state proxy metrics only

2.2 OTB-2015 Jogging (true ground truth)

Because vtest.avi cannot support accuracy claims, I downloaded the OTB-2015 **Jogging** sequence from the Visual Tracker Benchmark. OTB is a standard single-object tracking benchmark and includes per-frame rectangles. Jogging has 307 frames at 352x288 and 30 FPS, with two independently annotated runners in the same video. The runners pass behind a large pole, which is the occlusion event that motivates DriftGuard.

Property	Value
Dataset	OTB-2015 / Visual Tracker Benchmark (OTB100)
Sequence	Jogging
Authors / citation	Yi Wu, Jongwoo Lim, Ming-Hsuan Yang, TPAMI 2015
Resolution / FPS	352 x 288 @ 30 FPS
Frames	307
Ground truth	groundtruth_rect.1.txt and groundtruth_rect.2.txt
Target 1 initial box	(111, 98, 25, 101) — left runner
Target 2 initial box	(180, 79, 37, 114) — right runner
Local files	datasets/jogging_1.avi, datasets/jogging_1_gt.csv, datasets/jogging_2_gt.csv

Source archive used: the Wayback Machine copy of `http://cvlab.hanyang.ac.kr/tracker_benchmark/seq/Jogging.zip` (snapshot dated 2022-02-10). The original OTB site is no longer reliably available; the archived file is the one used by public OTB downloading scripts. The GT annotations and benchmark outputs are included for research and reproducibility; the video can be rebuilt with `download_otb_jogging.py`. Attribution is in `datasets/README.md`.

2.3 Ground-truth CSV format

Annotations are converted to a simple 0-based CSV so the same evaluator handles any annotated video:

```
frame,x,y,w,h
0,111,98,25,101
1,112,100,23,99
...
```

During evaluation the initial box is the first GT row. The tracker is then run one-pass without reinitialisation from GT. A box is a success at IoU threshold t when intersection-over-union with GT is at least t. Precision uses centre distance: a frame is correct at threshold d when the Euclidean distance between predicted and GT centres is at most d pixels.

2.4 Data acquisition failures and limitations

- No authoritative vtest.avi annotations were found in the OpenCV sample tree, OTB, or public tracking repositories. Creating synthetic labels for it would have made the evaluation circular, so it is not used for accuracy.
- OTB Jogging is a useful, small, standard dataset but only 307 frames. It is not a replacement for full OTB100, LaSOT, GOT-10k or TrackingNet evaluation.
- The OTB frames were converted to MJPG AVI for the current video pipeline. GT boxes are in original image coordinates; MJPG compression introduces only small pixel differences.
- Jogging contains camera pan and two similar-looking runners. This makes identity verification harder than a static-camera single-person clip and is a deliberate stress test.

3. Method

3.1 Runtime pipeline

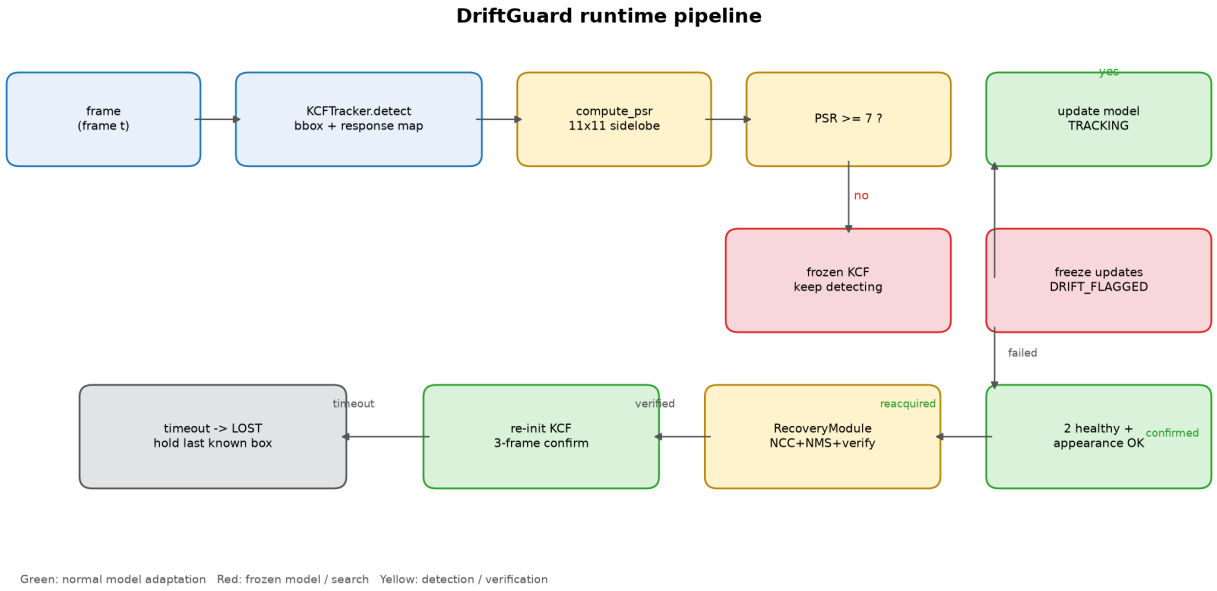


Figure 1. DriftGuard runtime pipeline. Green: normal adaptation. Red: frozen model/search. Yellow: detection and verification.

On every frame the tracker extracts a search patch around the current box and computes a correlation response. The response is evaluated with PSR. If confidence is high, the model is adapted and the box is drawn green. If confidence collapses, updates freeze, the box is drawn red, and recovery begins. Recovery first lets the frozen KCF attempt to reacquire the target; only if that fails does template re-detection run. A candidate must pass appearance and motion checks, and a re-initialised candidate must survive a confirmation window or the exact pre-drift model is restored.

3.2 Custom KCF core

For a windowed feature patch *x*, the Gaussian kernel correlation is:

```
k(x, z) = exp( -||x - z||^2 / (N * sigma^2) )
||x - z||^2 = ||x||^2 + ||z||^2 - 2 * IFFT(X * conj(Z))
alpha_f = Y_f / (K_f + lambda)
response = real( IFFT( alpha_f * FFT(k(x_model, z_search)) ) )
```

Y_f is the FFT of a centred 2-D Gaussian label. N is the feature-patch size and sigma controls kernel bandwidth. The ridge term lambda stabilises the frequency-domain solution. The complete response map is returned by `detect()`; this is the central reason we do not use OpenCV's KCF wrapper.

Features are grayscale plus the two Lab chroma channels, each normalised to [-0.5, 0.5] and multiplied by a separable Hann window. The chroma channels are a cheap appearance cue that helps distinguish a dark pedestrian from a dark pole or sign. The model patch is resized so its largest side is 96 pixels; padding is 2.5x the target box. Single-scale tracking is used because it is fast and predictable on the target sequences; scale adaptation remains a limitation.

3.3 PSR drift detection

For response R, let p = max(R) at (py, px). Exclude an 11x11 square centred on the peak and compute the mean and standard deviation of the remaining sidelobe:

```
PSR = (p - mean(R_sidelobe)) / (std(R_sidelobe) + epsilon)
```

A sharp, isolated peak gives high PSR; occlusion or a distractor spreads energy into sidelobes and PSR falls. The default threshold is 7.0. The 11x11 exclusion is the standard PSR convention and prevents the peak itself from inflating the sidelobe statistics.

3.4 Freeze, frozen reacquisition and template recovery

When PSR drops, no model update occurs. The system then keeps running `detect()` with the frozen model. If the target is only briefly occluded, the unchanged filter can re-lock after the occluder moves. Two consecutive PSR-healthy frames that also pass appearance plausibility are enough to resume adaptation. This is the **frozen-model reacquisition** path and it is what solved the Jogging-2 pole false-positive.

If frozen reacquisition does not occur within the debounce window (10 frames), `RecoveryModule` runs multi-template, multi-scale normalised cross-correlation in a search region around the last known good box. The region grows with lost-frame count. Up to three local peaks per template/scale are collected, greedy NMS keeps up to eight distinct candidates, and each candidate is verified by:

- NCC score from `TM_CCOEFF_NORMED` (minimum 0.50);
- global HSV histogram correlation plus upper/lower-body histograms (minimum 0.30 fused);
- mean SSIM against trusted reference patches (minimum 0.48);
- a weighted fusion score (NCC 0.55, colour 0.25, structure 0.20) and a minimum total of 0.66;
- a static-background difference check, so a sign/pole/pavement candidate is rejected;
- a motion gate that limits how far a candidate can jump while the target is occluded.

Accepted candidates are re-initialised into KCF and enter a three-frame confirmation window. During confirmation the model is frozen. If PSR or appearance fails, the pre-drift KCF snapshot is restored. If confirmation succeeds, the recovery is counted as confirmed and normal updates resume. After 40 lost frames the tracker enters `LOST` and holds the last known box rather than locking onto a random pedestrian.

3.5 Baseline tracker

The baseline uses the same custom KCF and detection code but updates its model on every frame, regardless of PSR. It has no freeze, no recovery, and no failure state. This isolates the effect of DriftGuard rather than comparing two different trackers.

Why PSR alone is not enough
PSR measures self-consistency, not identity. The baseline can have a high PSR while tracking background after contamination. The system therefore uses PSR to trigger freezing, but uses independent appearance verification and GT evaluation to decide whether a recovery is real.

4. Implementation

4.1 Classes and responsibilities

Component	Responsibility
KCFTracker	Feature extraction, Gaussian correlation, ridge-regression model, response map, translation update, model update, snapshot/restore.
compute_psr()	11x11 sidelobe exclusion, PSR scalar for each response map.
RecoveryModule	Reference bank, local/expanded multi-template search, NMS, HSV/SSIM/foreground verification, motion gate.
DriftGuard	State machine: tracking, drift, frozen reacquisition, template re-init, confirmation, rollback, LOST timeout.
BaselineKCF	Always-updating baseline using the same KCF core.
Evaluation / plotting	GT CSV parsing, success/precision curves, AUC, PSR plot, logs, JSON report.
CLI	Video/webcam input, bbox selection, thresholds, output paths, optional GT.

4.2 Important implementation details

- The response peak is subpixel-refined by quadratic interpolation before converting model-pixel displacement to image pixels.
- The KCF offset sign was initially inverted relative to the NumPy FFT correlation convention; this was detected on synthetic translation and fixed by negating the response peak displacement.
- A snapshot of the KCF Fourier model, ROI, template size, window and label is taken when a drift episode starts. It is restored if a candidate fails confirmation.
- Reference patches are added only on healthy frames and during confirmed recovery. This prevents occluders from entering the appearance bank.
- Colour histograms are computed over the whole box and over upper/lower halves. The lower-body term prevents a black jacket from making two different runners look identical.
- All thresholds are deliberately conservative. A missed recovery is preferred to a confident wrong identity switch.

4.3 Parameters used in the reported experiments

Parameter	Value	Purpose
PSR threshold	7.0	Drift flag / update freeze
KCF padding	2.5	Search region around target
KCF template size	96 px	Largest model patch side
KCF sigma	0.2	Gaussian kernel bandwidth
KCF lambda	1e-4	Ridge regularisation
KCF learning rate	0.075	Model interpolation
Recovery delay	10 frames	Give frozen KCF time to reacquire
Frozen reacquire confirmation	2 frames	Consecutive healthy + plausible
Template confirmation	3 frames	Probation before counting a recovery
Max lost frames	40	Timeout to LOST
NCC minimum	0.50	Template-match gate
Histogram minimum	0.30	Colour gate
SSIM minimum	0.48	Structure gate
Fusion minimum	0.66	Combined candidate score
Foreground minimum	8.0	Static-background difference
Recovery scales	0.9 / 1.0 / 1.1	Multi-scale template search
Reference bank	3 snapshots	Initial + recent appearance

4.4 Key code excerpt: frozen reacquisition

```
bbox_frozen, response = self.tracker.detect(frame)   # no update
psr_frozen = compute_psr(response)
healthy = (psr_frozen >= self.psr_threshold and
           self.recovery.is_plausible_track(frame, bbox_frozen))
if healthy:
    self._frozen_healthy_count += 1
else:
    self._frozen_healthy_count = 0
if self._frozen_healthy_count >= 2:
    self.tracker.update_model(frame, bbox_frozen)
    self.last_good_bbox = bbox_frozen
    self.state = self.RECOVERED
    self.successful_recoveries += 1
    return result
```

This path is deliberately first in the recovery order. It preserves the pre-occlusion model and avoids re-initialisation on a pole, while still allowing template matching when the target changes position or appearance too much for the frozen model.

4.5 Command-line interface and outputs

The main entry point is `driftguard.py`. Important options include `--source`, `--bbox`, `--psr-threshold`, `--recovery-delay-frames`, `--max-lost-frames`, `--gt` and `--metric-plot`. The program writes an annotated AVI, a PSR plot, an optional success/precision plot, a per-frame CSV log and a JSON report. The OTB benchmark is reproduced by `benchmark_otb.py`.

5. Experimental Protocol

5.1 One-pass evaluation

Both trackers are initialised from the first ground-truth box and run forward without any GT correction. For each frame we record the predicted box, PSR and state. At evaluation time we compare against GT where it exists. Baseline and DriftGuard use identical input frames, initial boxes and KCF core parameters; only the update/failure logic differs.

5.2 Metrics

Metric	Definition	Interpretation
IoU	|A ∩ B| / |A □ B|	Box overlap with ground truth
Success@0.5	Fraction of frames with IoU >= 0.5	Standard OTB success indicator
Success AUC	Mean success over IoU thresholds 0.00..1.00	Threshold-independent overlap quality
Mean IoU	Average IoU over all GT frames	Localisation quality
Precision@20px	Fraction with centre error <= 20 px	Centre accuracy
Precision AUC	Mean precision over 0..50 px	Threshold-independent centre accuracy
Recovery rate	Confirmed recoveries / drift episodes	Ability to recover after PSR collapse
Processing FPS	Frames / time spent in tracker+recovery	Computational efficiency; excludes decode/encode

5.3 Environment

Item	Value
OS	Windows 11 / win32
Python	3.12.10
OpenCV	5.0.0
NumPy	2.5.4
SciPy	1.18.1
Matplotlib	3.11.2
ReportLab	5.1.0
CPU	ordinary laptop-class CPU; reported FPS is indicative

FPS excludes video decode/encode and measures the tracking/recovery computation. Overall pipeline FPS includes both trackers plus OpenCV read/write and is the user-visible throughput. Hardware-dependent numbers are not a cross-machine benchmark.

6. Results

6.1 OTB Jogging accuracy

This is the primary result. Both targets have official GT and are evaluated over all 307 frames. The aggregate is frame-weighted over 614 annotated frames.

Sequence	Tracker	Success@0.5	Success AUC	Mean IoU	Precision@20px	Precision AUC
Jogging-1	DriftGuard	0.967	0.770	0.773	1.000	0.873
Jogging-1	Baseline	0.225	0.185	0.178	0.231	0.218
Jogging-2	DriftGuard	1.000	0.777	0.780	1.000	0.925
Jogging-2	Baseline	1.000	0.741	0.743	1.000	0.910
Aggregate	DriftGuard	0.984	0.773	0.776	1.000	0.899
Aggregate	Baseline	0.612	0.463	0.461	0.616	0.564

Main quantitative finding
DriftGuard reaches **0.984** success at IoU 0.5 and **1.000** precision at 20 px across both Jogging targets. The always-updating baseline reaches **0.612** success and **0.616** precision. The gain is driven by Jogging-1, where the baseline is destroyed by the pole occlusion.

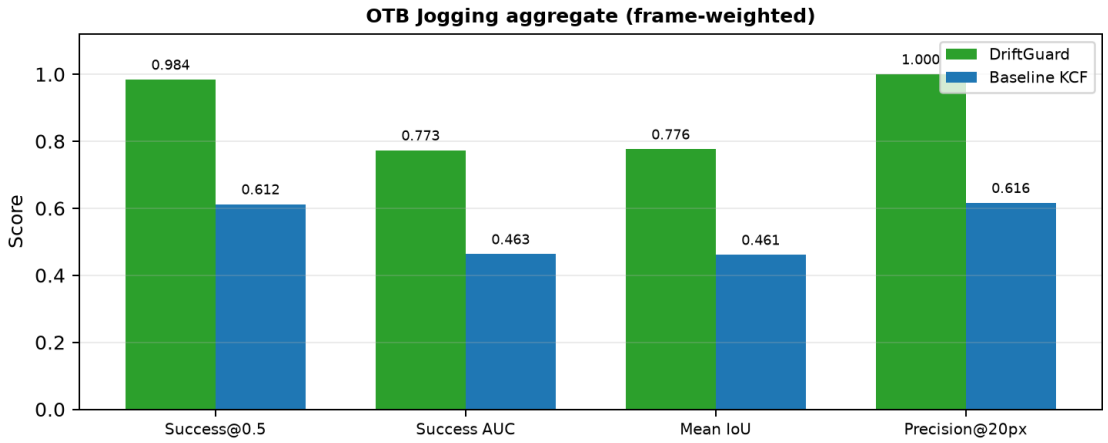


Figure 2. Frame-weighted aggregate metrics on the two OTB Jogging targets.

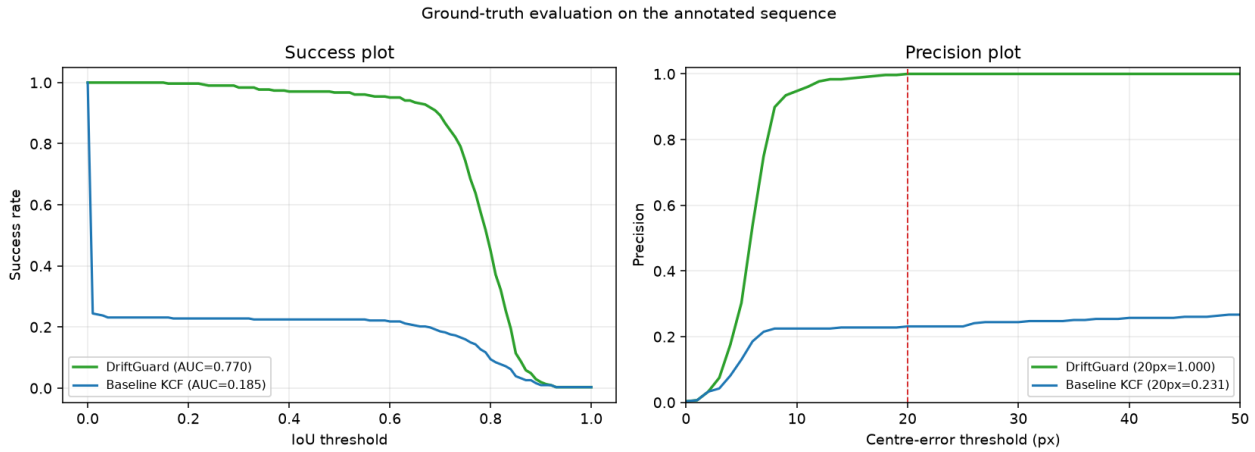


Figure 3. OTB success and precision curves, Jogging-1 (left runner). DriftGuard recovers after the pole occlusion; baseline permanently loses the target.

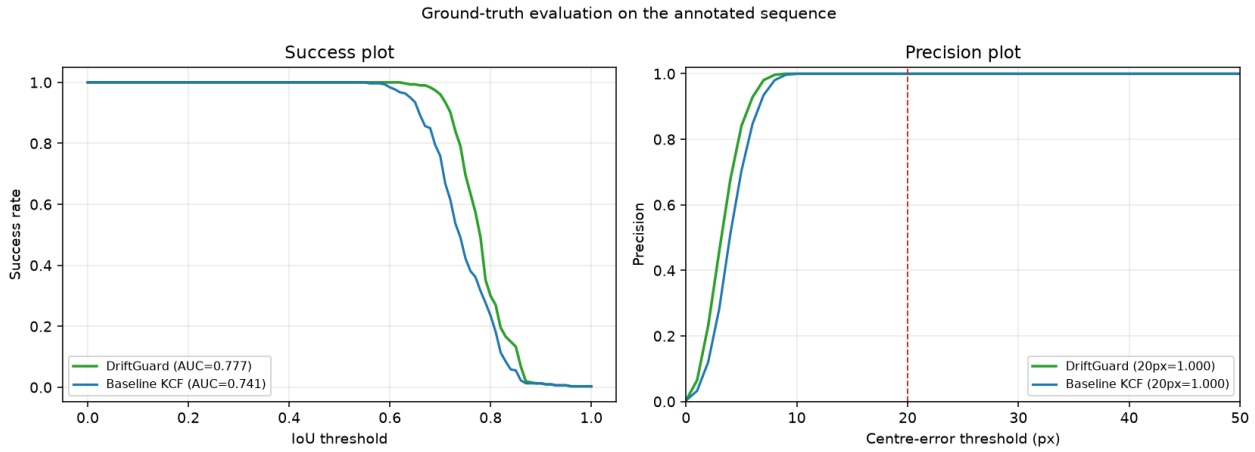


Figure 4. OTB success and precision curves, Jogging-2 (right runner). Both trackers succeed at 20 px, but DriftGuard has higher overlap AUC.

6.2 Recovery behaviour

Sequence	Drift episodes	Confirmed recoveries	Frozen reacquisition	Template re-init	Recovery rate	LOST frames
Jogging-1	1	1	0	1	100.0%	0
Jogging-2	1	1	1	0	100.0%	0

Jogging-1 uses template re-initialisation after the pole. Jogging-2 is recovered by the frozen filter before template search runs. Both are counted as confirmed only after appearance/PSR confirmation, and neither has any LOST frames.

6.3 Qualitative OTB frames



Figure 5. Jogging-1: initial tracking, drift at frame 67, template recovery at frame 82, resumed tracking at frame 140.



Figure 6. Jogging-2: drift at frame 50, frozen-KCF reacquisition at frame 57, resumed tracking at frame 120.

6.4 PSR behaviour

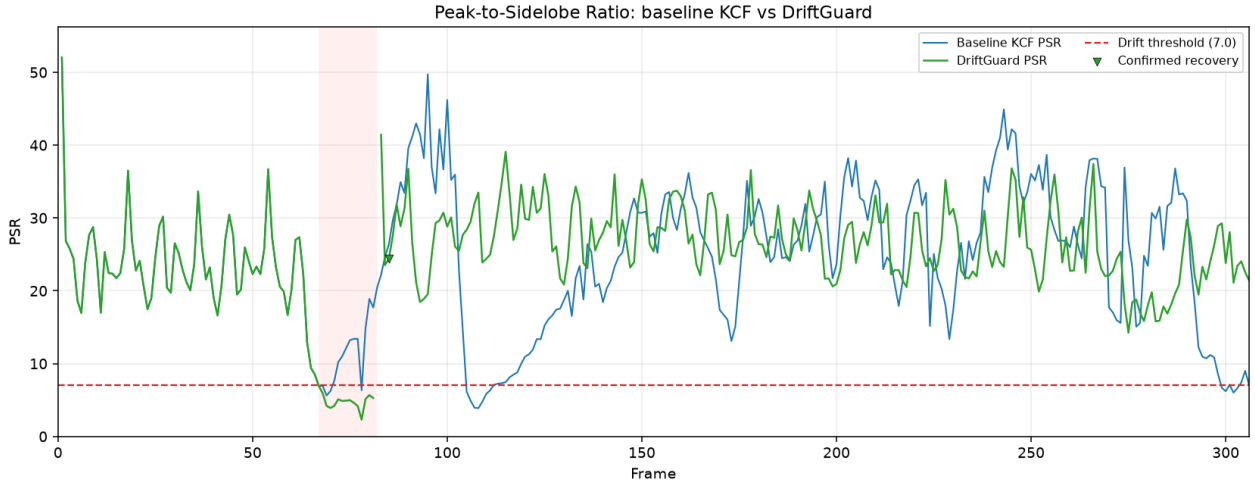


Figure 7. Jogging-1 PSR. The shaded interval is the drift episode; the green triangle marks confirmed recovery.

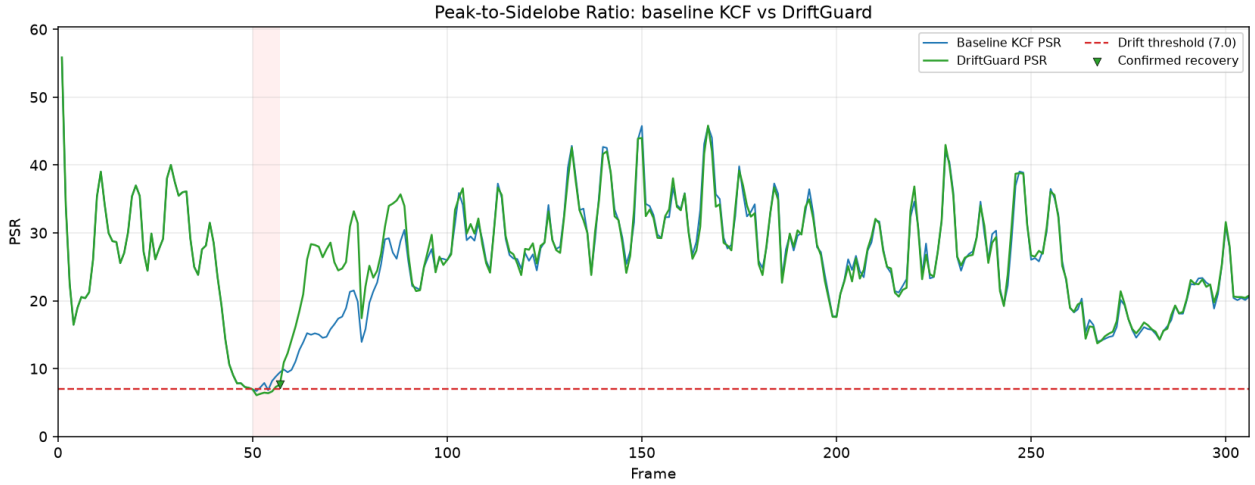


Figure 8. Jogging-2 PSR. Frozen reacquisition occurs at frame 57; no template re-initialisation is needed.

6.5 vtest.avi proxy results

vtest.avi has no GT. The following table is included for completeness but is not an accuracy claim. The 44.1% PSR-healthy value is caused by the system stopping adaptation after PSR collapse at the boundary; it is not comparable to the baseline 87% because the baseline keeps updating on whatever it last locked onto.

Proxy metric	DriftGuard	Baseline KCF
Frames processed	795	795
PSR-healthy frames	350 (44.1%)	691 (87.0%)
Drift-flagged frames	444	103
Drift episodes	2	13
Recovery attempts	2	n/a
Confirmed recoveries	1	n/a
Recovery success rate	50.0%	n/a
Frames held LOST	391	n/a
Model updates	348	n/a
Frozen updates	446	n/a
Mean processing FPS	408.2	531.3
Overall pipeline FPS	101.6	—



Figure 9. vtest.avi qualitative timeline: initial track, sign/pole drift at frame 37, template recovery at frame 52, and LOST timeout at frame 404.

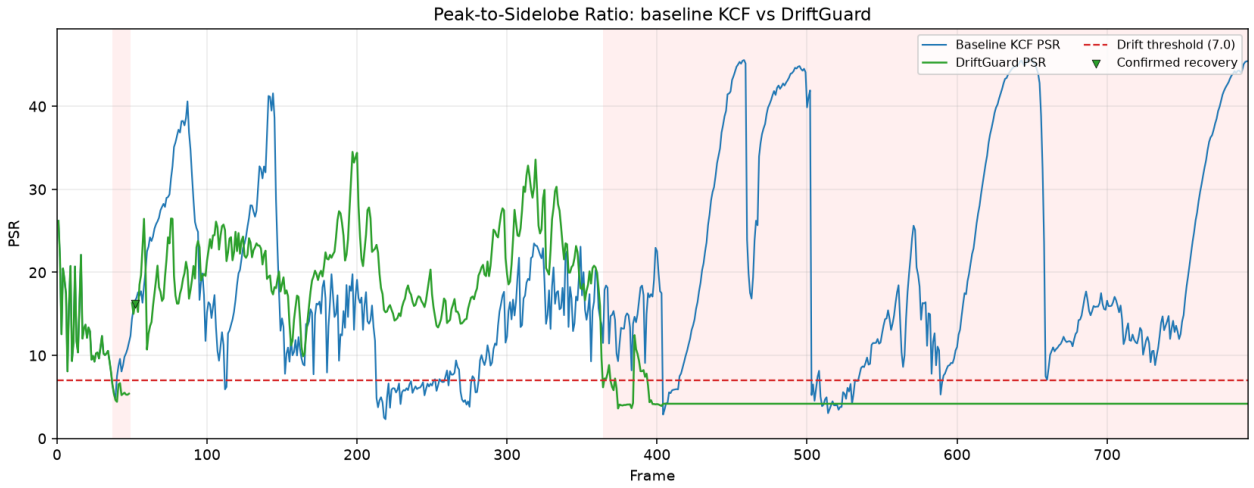


Figure 10. vtest.avi PSR comparison. The long shaded region starts at the frame-364 drift and becomes LOST at frame 404; it is not a ground-truth absence claim.

6.6 Computational efficiency

Run	DriftGuard processing FPS	Baseline processing FPS	Overall pipeline FPS	Source FPS
vtest.avi	408.2	531.3	101.6	10
Jogging-1	793.9	927.3	272.6	30
Jogging-2	826.6	763.9	258.8	30

The system is comfortably faster than both source frame rates. Recovery overhead is only paid during drift episodes, and the frozen reacquisition path is cheaper than template search because it reuses the existing filter.

7. Failed Attempts and Design Iteration

The final result was not the first result. This section records the approaches that failed, why they failed, and what replaced them. This is important because the vtest-only numbers looked much worse before the recovery state machine was fixed and because some apparently reasonable fixes made another sequence worse.

7.1 Missing vtest ground truth

The first report used PSR-healthy counts, drift counts and recovery rate as proxies. That produced an apparently poor 44.1% healthy result and a 50% recovery rate with no way to know whether the tracker was actually on the target. Attempts to find authoritative vtest annotations were unsuccessful, so I acquired OTB Jogging with official labels instead. The vtest proxy numbers are retained but explicitly labelled as non-accuracy metrics.

7.2 Response-map sign inversion

The first KCF implementation moved in the opposite direction of the target. The NumPy correlation convention produced a peak on the opposite side of the true translation. The issue was isolated with a synthetic diagonal-translation test and fixed by negating the peak offset. This is now covered by a regression test.

7.3 Sign/pole model contamination

With immediate template re-initialisation, the vtest tracker re-locked onto the sign at frame 37. The patch contained the target legs plus the sign/pole, passed loose verification, and then taught the KCF model the sign. PSR stayed high while the target left, which is the classic silent-contamination failure. Fixes included freezing updates, debouncing template search, foreground/static-background checks, appearance fusion and rollback.

7.4 False recovery on Jogging-2

With the earlier 3-frame recovery delay, Jogging-2 failed badly: the tracker re-initialised on the pole at frame 53. The patch was similar enough in raw appearance (NCC 0.727, histogram 0.944, SSIM 0.700) that verification accepted it, but the true target was hidden and the runner was later tracked incorrectly. This was fixed in two steps: (1) keep the frozen KCF detecting and give it a chance to reacquire, and (2) increase the template debounce.

7.5 The fixed-delay trap

A longer delay by itself fixes Jogging-2 because the target emerges from behind the pole before template search runs. But a long delay breaks vtest, where the target has moved and the search window is anchored at the old location. The table below shows the tuning sweep. The final design therefore cannot rely on delay alone: it must combine frozen reacquisition with a shorter 10-frame template delay.

Variant	Jogging-1 success@0.5	Jogging-2 success@0.5	vtest outcome	Decision
Baseline KCF (always update)	0.225	1.000	PSR-only; often contaminated	Reference
Delay 3	0.967	0.173	Recovery at frame 49	Rejected
Delay 5	0.967	0.179	Recovery at frame 49	Rejected
Delay 8	0.967	0.189	Recovery at frame 49	Rejected
Delay 10	0.967	0.189	Recovery at frame 49	Rejected alone
Delay 12	0.967	0.189	Recovery at frame 49	Rejected
Delay 15	0.967	0.189	Recovery at frame 49	Rejected
Delay 20	0.964	0.961	0 recoveries; 718 LOST frames	Rejected (vtest broken)
Final: frozen reacquire + delay 10	0.967	1.000	Recovery at frame 49	Selected

The final variant is the only one that keeps both Jogging targets at their best and preserves the vtest frame-49 recovery. This is why the report's main claim is based on the OTB benchmark, not on a single fixed delay.

7.6 Other rejected or deferred ideas

- **Full-frame template matching from the start:** too slow and too many distractors; rejected in favour of local search that expands with lost-frame count.
- **Haar/HOG people detector as re-detector:** would add a dependency/heavy detector and still needs identity verification; the template + appearance pipeline is much lighter.
- **Immediate model update after recovery:** risks learning a false candidate before confirmation; rejected in favour of a three-frame freeze probation.
- **Single global PSR threshold without appearance check:** allowed confident pole/background locks; rejected.
- **Multi-scale KCF:** considered but not implemented in this version. The fixed box still reaches ~0.78 mean IoU on Jogging; scale adaptation remains future work.
- **vtest manual annotation by the tracker itself:** circular and not valid; rejected. OTB GT was used instead.

7.7 Remaining limitations

- The quantitative benchmark is small: OTB Jogging, 307 frames, two targets. It is not a full OTB100/LaSOT evaluation.
- The tracker is single-scale. Large apparent-size changes may reduce overlap.
- The static-background difference cue assumes a mostly fixed camera; it is a supporting cue, not the primary recovery mechanism.
- Template appearance is still hand-crafted (NCC + HSV + SSIM), not a learned re-identification embedding.
- vtest.avi has no GT, so its outcome remains qualitative despite the visual recovery at frame 52.
- The real-time FPS numbers are machine-dependent and include the custom NumPy KCF, not a compiled C++ implementation.

8. Conclusion

The final system is a defensible lightweight DriftGuard implementation: it exposes the KCF response, detects confidence collapse with PSR, freezes adaptation, gives the frozen filter a chance to reacquire, and only then runs verified template recovery. The critical engineering fix was to treat recovery as a stateful, verifiable process rather than an immediate re-initialisation. The critical evaluation fix was to stop using vtest proxies as accuracy and evaluate on a dataset with official ground truth.

On OTB Jogging, DriftGuard reaches 0.984 aggregate success at IoU 0.5 and 1.000 precision at 20 px, versus 0.612 and 0.616 for the always-updating baseline. Both occlusion episodes are recovered and neither Jogging run enters LOST. The result is not evidence of state-of-the-art tracking, but it is a clear, reproducible demonstration that freeze + verified recovery beats unconditional model update on the occlusion cases this project targets.

Recommended next steps

1) Add scale estimation (DSST/SAMF-style) and evaluate on a larger OTB100 subset. 2) Replace hand-crafted verification with a lightweight person re-ID embedding. 3) Add a moving-camera-compatible foreground/motion model. 4) Evaluate on LaSOT/GOT-10k and report attribute-wise results. 5) Add a true long-term target-absence protocol rather than only a timeout state.

9. Reproducibility

9.1 Installation

```
python -m pip install -r requirements.txt
```

9.2 Original vtest demo

```
python driftguard.py --download
python driftguard.py --source vtest.avi
python driftguard.py --source vtest.avi --display
```

9.3 OTB Jogging benchmark

```
python download_otb_jogging.py
python benchmark_otb.py

# or individually
python driftguard.py --source datasets/jogging_1.avi \
  --bbox 111,98,25,101 --gt datasets/jogging_1_gt.csv \
  --metric-plot datasets/jogging1_success_precision.png

python driftguard.py --source datasets/jogging_1.avi \
  --bbox 180,79,37,114 --gt datasets/jogging_2_gt.csv \
  --metric-plot datasets/jogging2_success_precision.png
```

9.4 Tests

```
python -m unittest discover -s tests -v
```

Seven tests pass: PSR on peaked/flat maps, subpixel peak direction, IoU, GT success/precision/AUC, and synthetic diagonal-translation tracking with the custom KCF. The report generation itself is separate from the runtime system.

9.5 Output manifest

Path	Contents
driftguard.py	Complete implementation and CLI
benchmark_otb.py	Reproduces the two-target OTB Jogging benchmark
download_otb_jogging.py	Downloads the sequence and writes conversion/GT files
make_report.py, report_assets/	Builds this PDF and stores its figures
DriftGuard_Technical_Report.pdf	This report
driftguard_output.avi	Annotated vtest output
psr_comparison.png	vtest PSR comparison
preview.jpg	Annotated vtest recovery frame
datasets/jogging_1.avi	OTB Jogging frames
datasets/jogging_1_gt.csv, jogging_2_gt.csv	Official OTB annotations converted to CSV
datasets/jogging1_output.avi, jogging2_output.avi	Annotated OTB outputs
datasets/jogging1_psr.png, jogging2_psr.png	OTB PSR plots
datasets/jogging1_success_precision.png, jogging2_success_precision.png	OTB success/precision curves
datasets/jogging1_report.json, jogging2_report.json	Full OTB result reports
datasets/benchmark_summary.json	Frame-weighted aggregate
tests/test_driftguard.py	Unit/smoke tests

Appendix A. Metric Definitions

Let A be the predicted box and B the ground-truth box. IoU is the area of intersection divided by the area of union. Success at threshold t is the fraction of frames with IoU >= t. Success AUC is the mean success rate over 101 thresholds from 0 to 1. Centre error is the Euclidean distance between the predicted and GT box centres. Precision at 20 px is the fraction of frames with centre error <= 20 pixels. Precision AUC is the mean precision over 51 distance thresholds from 0 to 50 px.

These are OTB-style one-pass evaluation metrics. No tracker is restarted from GT after failure. This is deliberately harsh: a tracker that loses the target receives zero-or-low overlap for the remaining frames unless it recovers on its own.

Appendix B. Complete Result Summary

The following values are copied from the checked-in JSON reports.

Run	Frames	Drift frames	Episodes	Recoveries	Frozen recoveries	LOST frames	Success@0.5	Precision@20
vtest.avi	795	444	2	1	0	391	n/a	n/a
Jogging-1	307	16	1	1	0	0	0.967	1.000
Jogging-2	307	8	1	1	1	0	1.000	1.000

Appendix C. References and Data Attribution

[1] J. F. Henriques, R. Caseiro, P. Martins, J. Batista. “High-Speed Tracking with Kernelized Correlation Filters.” IEEE TPAMI, 2015.

[2] Y. Wu, J. Lim, M.-H. Yang. “Object Tracking Benchmark.” IEEE TPAMI, 2015. OTB-2015 / Visual Tracker Benchmark.

[3] OpenCV sample data: `opencv/4.x/samples/data/vtest.avi`.

[4] OTB Jogging archive used: Wayback Machine snapshot `web.archive.org/web/20220210102816/http://cvlab.hanyang.ac.kr/tracker_benchmark/seq/Jogging.zip`.

[5] Public OTB archive helper referenced for the Wayback URL: `github.com/shinlnh/OTB100_Dataset` (download script only; no tracker code copied).

This report describes the implementation and experiments performed in this workspace. The included OTB Jogging data is attributed to its original authors and is retained for research/reproducibility. The vtest clip is an OpenCV sample.